

Documento de Requisitos de Software

Sistema de Gestión de Incidencias con Enrutamiento Inteligente

(Algoritmo de clasificación por reglas + Auditoría de Seguridad)

Curso de Ingeniería de Software — Sexto Semestre

Versión 3.0

1. Introducción y planteamiento del problema

Los sistemas de gestión de incidencias (tickets) dependen habitualmente de que el propio usuario clasifique manualmente su solicitud, indicando categoría y prioridad, o de que un agente lo redirija manualmente al área correspondiente. Esta clasificación manual introduce errores de enrutamiento, retrasos en la atención y sobrecarga sobre el usuario final y los agentes.

Adicionalmente, la mayoría de estos sistemas no incorporan mecanismos sistemáticos de auditoría sobre quién accede a qué información dentro de la propia plataforma, ni distinguen el nivel de urgencia real según quién reporta el problema.

El presente proyecto propone el desarrollo de una plataforma de gestión de incidencias que integra tres componentes diferenciadores: (1) un algoritmo determinístico de clasificación por reglas que asigna categoría, área responsable y prioridad a cada ticket, incorporando el nivel del solicitante en el cálculo de la prioridad final; (2) un mecanismo de plazos de atención (SLA) con recordatorios automáticos; y (3) un módulo de auditoría de seguridad que registra de forma sistemática los accesos y acciones críticas realizadas sobre el sistema.

Como parte del proceso de diseño, se evaluó inicialmente un enfoque de clasificación basado en un modelo de lenguaje local (Ollama). Los hallazgos de esa evaluación y la justificación del cambio hacia un algoritmo de reglas se documentan en la sección 9.6 (Spike técnico).

2. Objetivos

2.1 Objetivo general

Desarrollar una plataforma de gestión de incidencias que clasifique y enrute automáticamente los tickets mediante un algoritmo de reglas, gestione plazos de atención según prioridad y nivel del solicitante, e incorpore un módulo de auditoría de seguridad sobre los accesos al sistema.

2.2 Objetivos específicos

1. Implementar un módulo de autenticación y autorización con roles diferenciados (administrador, agente, usuario final).
2. Implementar el ciclo de vida completo de un ticket (creación, asignación, cambio de estado, comentarios, cierre).
3. Diseñar e implementar un algoritmo de reglas que clasifique cada ticket por categoría, lo enrute al área de soporte correspondiente y le asigne una prioridad base.
4. Ajustar la prioridad final del ticket considerando el nivel del solicitante (usuario VIP/ejecutivo vs. usuario normal).
5. Calcular un plazo límite de atención (SLA) según la prioridad final y notificar mediante recordatorios cuando un ticket esté próximo a vencer.
6. Registrar de forma inmutable los accesos y acciones críticas ejecutadas sobre el sistema.
7. Detectar y mitigar intentos de acceso no autorizado mediante bloqueo por intentos fallidos repetidos.
8. Medir y presentar métricas de desempeño del sistema mediante un panel para el administrador.

3. Alcance del proyecto

Dada la duración del semestre y la composición del equipo de trabajo (tres integrantes, dos de ellos asignados por el docente sin experiencia previa confirmada en desarrollo), el proyecto se divide en un alcance mínimo viable (MVP), de entrega obligatoria, y un conjunto de extensiones que se incorporarán si el tiempo disponible lo permite.

3.1 Alcance mínimo viable (MVP)

- Autenticación de usuarios con roles y almacenamiento seguro de contraseñas (hash).
- Gestión de tickets: creación, asignación, cambio de estado y comentarios.
- Algoritmo de reglas que clasifica cada ticket por categoría, área de soporte y prioridad base al momento de su creación.
- Ajuste de prioridad por nivel del solicitante (VIP vs. normal).

3.2 Extensiones (sujetas a disponibilidad de tiempo)

- Cálculo de SLA y recordatorios automáticos de vencimiento, visibles para el creador y el agente asignado.
- Módulo de auditoría: registro inmutable de accesos y acciones críticas.
- Bloqueo por intentos de inicio de sesión fallidos repetidos (mitigación de fuerza bruta).
- Panel de métricas (dashboard) para el administrador: tickets por estado, área y tiempo promedio de resolución.

3.3 Fuera de alcance

- Despliegue en infraestructura de producción o en la nube.
- Integración con sistemas de terceros (correo electrónico, mensajería, etc.).
- Uso de modelos de lenguaje (IA) en el sistema en producción; su evaluación quedó documentada como spike técnico, no como componente del sistema final.
- Modelado detallado de la estructura organizacional completa de la empresa (departamentos como marketing, producción, etc.); solo se distingue el nivel del solicitante (normal/VIP) en la medida en que afecta la prioridad.

4. Requisitos funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe permitir el registro e inicio de sesión de usuarios con roles diferenciados.	Alta
RF-02	El sistema debe permitir a un usuario final crear un ticket describiendo su incidencia en texto libre.	Alta
RF-03	El sistema debe clasificar automáticamente cada ticket entrante por categoría y área de soporte mediante un algoritmo de reglas basado en palabras clave.	Alta
RF-04	El sistema debe asignar una prioridad base al ticket según su categoría.	Alta
RF-05	El sistema debe elevar la prioridad de un ticket a atención inmediata cuando el solicitante tiene nivel VIP/ejecutivo, independientemente de la categoría.	Alta
RF-06	El sistema debe calcular una fecha límite de atención (SLA) a partir de la prioridad final del ticket.	Media
RF-07	El sistema debe generar un recordatorio visible para el creador y el agente asignado cuando un ticket esté próximo a vencer su SLA.	Media

RF-08	El sistema debe permitir a un agente visualizar los tickets de su área, ordenados por prioridad.	Alta
RF-09	El sistema debe permitir a un agente cambiar el estado de un ticket y añadir comentarios.	Alta
RF-10	El sistema debe registrar en un log de auditoría cada acceso y acción crítica (creación, cambio de estado, cierre).	Media
ID	Descripción	Prioridad
RF-11	El sistema debe bloquear temporalmente una cuenta tras un número configurable de intentos fallidos de inicio de sesión.	Media
RF-12	El sistema debe presentar un panel con métricas agregadas de tickets para el administrador.	Baja

5. Requisitos no funcionales

ID	Descripción	Categoría
RNF-01	Las contraseñas deben almacenarse utilizando una función de hash con sal (bcrypt o equivalente).	Seguridad
RNF-02	El registro de auditoría no debe ser editable ni eliminable desde la interfaz de la aplicación.	Seguridad
RNF-03	La clasificación de un ticket mediante el algoritmo de reglas debe ser instantánea (sin dependencia de servicios externos).	Rendimiento
RNF-04	El sistema debe ejecutarse en un entorno local sin dependencias de servicios en la nube de pago.	Portabilidad
RNF-05	El código debe seguir una estructura modular que separe rutas, lógica de negocio y acceso a datos.	Mantenibilidad
RNF-06	Las reglas de clasificación (palabras clave por categoría) deben estar centralizadas en un solo lugar del código, para facilitar su ajuste sin modificar la lógica del algoritmo.	Mantenibilidad

6. Casos de uso principales

ID	Actor	Descripción
CU-01	Usuario final	Crear un nuevo ticket describiendo una incidencia.
CU-02	Sistema	Clasificar, enrutar y priorizar automáticamente el ticket recién creado.
CU-03	Sistema	Calcular el SLA y emitir recordatorios de vencimiento.
CU-04	Agente	Consultar los tickets de su área, ordenados por prioridad.

CU-05	Agente	Actualizar el estado de un ticket y agregar comentarios.
CU-06	Administrador	Consultar el registro de auditoría de accesos y acciones.
CU-07	Administrador	Consultar el panel de métricas del sistema.

7. Backlog inicial — historias de usuario

1. Como usuario final, quiero crear un ticket describiendo mi problema, para que sea atendido por el equipo de soporte.
2. Como sistema, quiero clasificar y enrutar automáticamente cada ticket por categoría y área, para reducir errores de enrutamiento manual.
3. Como sistema, quiero elevar la prioridad de un ticket cuando el solicitante es VIP/ejecutivo, para reflejar la urgencia real del negocio.
4. Como sistema, quiero calcular un plazo de atención y avisar cuando esté por vencer, para que ningún ticket se quede sin respuesta.
5. Como agente, quiero ver los tickets de mi área ordenados por prioridad, para atender primero los casos más urgentes.
6. Como administrador, quiero ver un registro de quién accedió o modificó cada ticket y cuándo, para poder auditar el uso del sistema.
7. Como administrador, quiero que el sistema bloquee cuentas tras varios intentos fallidos de inicio de sesión, para mitigar ataques de fuerza bruta.

8. Arquitectura y stack tecnológico

8.1 Componentes

- Backend: Python con Flask, exponiendo la lógica de negocio y las rutas de la aplicación.
- Base de datos: SQLite.
- Algoritmo de clasificación: módulo propio en Python basado en coincidencia de palabras clave por categoría, sin dependencias externas ni modelos de IA en producción.
- Frontend: plantillas HTML/CSS renderizadas por Flask (Jinja2), sin framework de frontend adicional.
- Control de versiones: Git, con repositorio alojado en GitHub y flujo de trabajo basado en ramas por funcionalidad.

8.2 Justificación

Se prioriza un stack simple y ya conocido por el equipo, evitando sobre-ingeniería y dependencias externas frágiles (como un servicio de IA), dado el tamaño del equipo y la duración limitada del semestre.

8.3 Modelo de datos

Entidad	Campos principales
Usuario	id, nombre, email, contraseña_hash, rol (final/agente/admin), área_soporte (si es agente), nivel (normal/VIP), intentos_fallidos, bloqueado_hasta

Ticket	id/radicado, texto, categoría, prioridad, estado, creador_id, agente_id, fecha_creación, fecha_límite, fecha_cierre
Comentario	id, ticket_id, autor_id, texto, fecha
LogAuditoria	id, usuario_id, acción, detalle, fecha

8.4 Clases de servicio (lógica de negocio)

Siguiendo el principio de responsabilidad única, la lógica de negocio se separa de las entidades de datos en clases de servicio independientes. Ninguna entidad (Usuario, Ticket) contiene lógica de negocio propia; solo almacena estado.

Clase de servicio	Responsabilidad
ServicioAutenticacion	Registro de usuarios, verificación de credenciales, generación de hash de contraseña y control de bloqueo por intentos fallidos (RF-11). Es la única clase autorizada a manejar contraseñas en texto plano.
Clase de servicio	Responsabilidad
ClasificadorTickets	Determina la categoría de un ticket a partir de su texto, mediante coincidencia de palabras clave (sección 8.5). El área de soporte se deriva directamente de la categoría (relación 1 a 1); la prioridad base la asigna ServicioTickets según la categoría ya determinada.
GestorSLA	Calcula la fecha límite de atención según la prioridad final del ticket y el nivel del usuario (VIP tiene una tabla de horas propia, más corta, no solo una prioridad más alta), y verifica cuáles tickets están vencidos o próximos a vencer.
ServicioTickets	Orquesta el ciclo de vida del ticket: creación (aplica clasificación, prioridad base y SLA), listar por área, cambio de estado, reasignación de agente y comentarios.
ServicioAuditoria	Registra cada acción crítica del sistema (login, registro, tickets, comentarios, bloqueos) en el log de auditoría, con reintento automático ante fallos de guardado.
ServicioMetricas	Calcula los conteos agregados que alimenta el panel de métricas del administrador (tickets por estado, categoría y prioridad, vencidos y próximos a vencer).

8.5 Categorías de clasificación

El algoritmo de reglas clasifica cada ticket en una de las siguientes categorías, definidas para un escenario de soporte de TI interno de una empresa genérica:

- Infraestructura (hardware)
- Redes
- Permisos
- Cuentas y contraseñas
- Hackeos/Seguridad
- Software/Aplicaciones
- Otros/General (categoría por defecto cuando no hay coincidencia clara)

Dado que varias categorías comparten palabras clave (por ejemplo, "acceso" aparece tanto en Permisos como en Cuentas y contraseñas), el algoritmo define reglas de desempate explícitas antes de su implementación, documentadas junto al código.

9. Metodología de desarrollo

El proyecto se desarrolla bajo el marco de trabajo Scrum, complementado con un enfoque de prototipado evolutivo (spike) para reducir la incertidumbre técnica antes de comprometer decisiones de diseño en el plan de sprints.

9.1 Roles Scrum

Rol	Asignación	Responsabilidad
Product Owner	Definido junto al docente	Prioriza el backlog y valida los criterios de aceptación de cada historia.
Scrum Master	Desarrollador principal	Facilita las ceremonias, elimina bloqueos y da seguimiento al avance del sprint.
Dev Team	Los tres integrantes	Ejecuta las tareas del sprint backlog. Ver sección 11 sobre reparto real de trabajo.

9.2 Artefactos

- Product Backlog: lista priorizada de historias de usuario (sección 7).
- Sprint Backlog: subconjunto de historias comprometidas para cada sprint de 2 semanas.
- Incremento: versión funcional del sistema al cierre de cada sprint, desplegable localmente.

9.3 Ceremonias

Ceremonia	Frecuencia	Propósito
Sprint Planning	Inicio de cada sprint	Seleccionar y estimar las historias del sprint backlog.
Daily Standup	Diaria (asíncrona, por chat de equipo)	Reportar avance, plan del día y bloqueos.
Sprint Review	Cierre de cada sprint	Demostrar el incremento funcional al equipo y, si aplica, al docente.
Sprint Retrospective	Cierre de cada sprint	Identificar qué mejorar en el proceso de trabajo del equipo, incluyendo el cumplimiento de tareas asignadas.

9.4 Definition of Ready (DoR)

- Tiene criterios de aceptación claros y verificables.
- Su tamaño permite completarla dentro de un único sprint.
- No depende de una historia que aún no se ha completado.

9.5 Definition of Done (DoD)

- El código está implementado, integrado a la rama principal y libre de errores conocidos.
- Fue probada manualmente contra sus criterios de aceptación.
- Cuenta con comentarios claros en el código.
- La documentación relacionada (requisitos, UML) fue actualizada si hubo cambios.

9.6 Spike técnico — evaluación de clasificación por IA

Antes de definir el diseño final del módulo de clasificación, se ejecutó un spike de tiempo acotado para evaluar la viabilidad de clasificar tickets mediante un modelo de lenguaje local (Ollama, modelo llama3), en lugar de un algoritmo de reglas fijas.

9.6.1 Metodología del spike

Se definió un conjunto de 5 tickets de prueba con categoría y prioridad esperadas conocidas de antemano. Cada ticket se envió al modelo mediante un prompt que solicitaba una respuesta estructurada en formato JSON, y se midió: (1) si la respuesta era JSON válido, (2) si la categoría asignada coincidía con la esperada, (3) el tiempo de respuesta.

9.6.2 Resultados

Métrica	Resultado
JSON válido	5 de 5 tickets (100%)
Tiempo de respuesta promedio	~3.6 segundos por ticket
Consistencia de nombres de categoría	Baja: el modelo generó etiquetas distintas para conceptos equivalentes en distintos tickets (por ejemplo, "Sistema" y "Sistemas" como categorías separadas para casos similares).
Precisión de prioridad	3 de 5 tickets coincidieron con la prioridad esperada; 2 casos fueron sobre-priorizados por el modelo sin justificación clara en el texto del ticket.

9.6.3 Decisión

Se descartó el uso de un modelo de lenguaje para la clasificación en producción, por dos razones respaldadas por los resultados del spike: (1) la falta de un conjunto cerrado de categorías genera etiquetas inconsistentes, lo cual es inviable para un sistema que necesita agrupar y filtrar tickets de forma confiable; y (2) la variabilidad en la asignación de prioridad introduce comportamiento no determinístico difícil de auditar. En su lugar, se optó por un algoritmo de reglas basado en palabras clave, con categorías y criterios de prioridad fijos y verificables, evitando además una dependencia externa (Ollama) que el sistema en producción no necesita.

10. Plan de trabajo (sprints)

Sprint	Duración	Entregable principal (desarrollador principal)
Spike	Completado	Evaluación de clasificación por IA (sección 9.6).
Sprint 1	2 semanas	Modelo de base de datos, autenticación con roles y CRUD base de tickets.

Sprint 2	2 semanas	Algoritmo de clasificación por reglas, ajuste de prioridad por nivel VIP, cálculo de SLA y recordatorios.
Sprint 3	2 semanas	Módulo de auditoría, bloqueo por fuerza bruta y panel de métricas.
Sprint 4	2 semanas	Integración final, corrección de errores y documentación.

11. Roles del equipo y reglas de trabajo

Integrante	Responsabilidad principal
Desarrollador principal	Arquitectura, backend, algoritmo de clasificación, SLA y lógica de negocio (ruta crítica del proyecto en todos los sprints).
Segundo integrante	Tareas asignadas por sprint sin dependencia crítica: pruebas manuales, documentación de UML, plantillas de interfaz.
Tercer integrante	Tareas asignadas por sprint sin dependencia crítica: pruebas manuales, documentación de UML, plantillas de interfaz.

11.1 Reglas de trabajo en equipo

- Toda tarea asignada a un integrante queda registrada por escrito en el tablero del equipo, con responsable y fecha límite visibles.
- Todos los integrantes deben registrar su trabajo mediante commits propios en el repositorio compartido; el historial de Git es la evidencia objetiva de aporte individual.
- Ninguna tarea de la ruta crítica del proyecto depende de un integrante distinto al desarrollador principal, para evitar bloqueos por incumplimiento.
- El incumplimiento de una tarea asignada se documenta en la retrospectiva del sprint correspondiente.